

Lab 10 - Expected verification evidence

Run from the lab directory, after the failure injection in section 6 (with cargo-modeller deleted again) and before cleanup:

```
bash verification/checks.sh
```

Expected transcript - stock kind cluster, no metrics-server (the default path)

```
Lab 10 verification - namespace kcna-lab10

== Namespace and workload profiles
[PASS] namespace kcna-lab10 is Active (= Active)
[PASS] workload profiles read: 6 rows

== QoS class is derived by the API server, not declared
[PASS] berth-ledger qosClass (= Guaranteed)
[PASS] rate-quoter qosClass (= Burstable)
[PASS] notice-board qosClass (= BestEffort)
[PASS] ledger-compactor qosClass (= Guaranteed)
[PASS] quote-engine qosClass (= Burstable)

== Requests and limits landed as written in the profile
[PASS] berth-ledger cpu request/limit (= 100m/100m)
[PASS] rate-quoter memory request/limit (= 64Mi/192Mi)
[PASS] notice-board declares no requests and no limits (= )

== OOMKill evidence
[PASS] ledger-compactor termination reason (= OOMKilled)
[PASS] ledger-compactor exit code (128 + SIGKILL 9) (= 137)
[PASS] ledger-compactor memory limit that caused it (= 32Mi)

== LimitRange
[PASS] LimitRange defaultRequest.cpu (= 50m)
[PASS] LimitRange default.cpu (= 500m)
[PASS] LimitRange max.cpu (= 1)

== ResourceQuota
[PASS] quota hard requests.cpu (= 1)
[PASS] quota used requests.cpu is being tracked (= 350m)
[PASS] over-budget Pod cargo-modeller was NOT created (= )

== Deployment and HPA
[PASS] quote-engine readyReplicas (= 2)
[PASS] HPA served as autoscaling/v2 (= autoscaling/v2)
[PASS] HPA minReplicas (= 2)
[PASS] HPA maxReplicas (= 6)
[PASS] HPA cpu target utilisation (= 60)

== Metrics pipeline - reported honestly, not assumed
[note] metrics-server is NOT installed (v1beta1.metrics.k8s.io: absent) - this is the stock
kind default
[PASS] HPA correctly reports it cannot read metrics (ScalingActive
reason='FailedGetResourceMetric') - the pipeline is missing, not the HPA spec

-----
Lab 10: 25 passed, 0 failed
ALL CHECKS PASSED
```

```
echo $?
```

```
0
```

Alternative transcript - with the optional metrics-server installed

Only the final block changes:

```
== Metrics pipeline - reported honestly, not assumed
[noted] metrics-server IS installed (v1beta1.metrics.k8s.io Available=True)
[PASS] HPA ScalingActive reason with metrics present: 'ValidMetricFound'
```

```
-----
Lab 10: 25 passed, 0 failed
ALL CHECKS PASSED
```

The pass count is identical. The script branches on the real state of the API registry; it never asserts that kubectl top worked.

Supporting evidence the learner should be able to produce

1. All three QoS classes, side by side, derived by the API server

```
kubectl -n kcna-lab10 get pods \
-o custom-columns=POD:.metadata.name,QOS:.status.qosClass,CPU_REQ:.spec.containers[0].
resources.requests.cpu,CPU_LIM:.spec.containers[0].resources.limits.cpu,MEM_REQ:.spec.
containers[0].resources.requests.memory,MEM_LIM:.spec.containers[0].resources.limits.memory
```

POD	QOS	CPU_REQ	CPU_LIM	MEM_REQ	MEM_LIM
berth-ledger	Guaranteed	100m	100m	128Mi	128Mi
notice-board	BestEffort	<none>	<none>	<none>	<none>
rate-quoter	Burstable	50m	200m	64Mi	192Mi

The word qosClass appears in none of those manifests.

2. The OOMKill

```
kubectl -n kcna-lab10 get pod ledger-compactor \
-o jsonpath='reason={.status.containerStatuses[0].state.terminated.reason} exitCode={.status.
containerStatuses[0].state.terminated.exitCode} limit={.spec.containers[0].resources.limits.
memory} qos={.status.qosClass}'
```

```
reason=OOMKilled exitCode=137 limit=32Mi qos=Guaranteed
```

and the truncated log - the completion line never printed:

```
compactor: memory limit is 32Mi; attempting a 64 MiB working set
```

3. The LimitRange rewriting a Pod at admission

```
kubectl -n kcna-lab10 get pod notice-board -o jsonpath='notice-board(pre-LimitRange)={.status.
qosClass}'; echo
kubectl -n kcna-lab10 get pod late-arrival -o jsonpath='late-arrival(post-LimitRange)={.
status.qosClass}'; echo
```

```
notice-board(pre-LimitRange)=BestEffort
late-arrival(post-LimitRange)=Burstable
```

4. The quota rejection - an admission failure, not a scheduling failure

```
Error from server (Forbidden): error when creating "manifests/60-pod-quota-violation.yaml":
pods "cargo-modeller" is forbidden: exceeded quota: mfs-lab-quota, requested: requests.
cpu=900m, used: requests.cpu=350m, limited: requests.cpu=1
```

```
kubectl -n kcna-lab10 get pod cargo-modeller
```

```
Error from server (NotFound): pods "cargo-modeller" not found
```

5. The LimitRange rejection - a different plugin, a per-container bound

```
Error from server (Forbidden): pods "oversized" is forbidden: maximum cpu usage per Container
is 1, but limit is 2
```

6. The HPA, read honestly

```
kubectl -n kcna-lab10 get hpa quote-engine \
-o jsonpath='{range .status.conditions[*]}{.type}={.status} reason={.reason}{"\n"}{end}'
```

```
AbleToScale=True reason=SucceededGetScale
ScalingActive=False reason=FailedGetResourceMetric
```

with the summary line:

NAME	REFERENCE	TARGETS	MINPODS
MAXPODS	REPLICAS	AGE	
quote-engine	Deployment/quote-engine	cpu: <unknown>/60%, memory: <unknown>/200Mi	2
6	2	20s	

and the underlying reason, from the API registry rather than from inference:

```
kubectl get apiservice v1beta1.metrics.k8s.io
```

```
Error from server (NotFound): apiservices.apiregistration.k8s.io "v1beta1.metrics.k8s.io" not
found
```

Notes on variance

* used: requests.cpu=350m in the quota rejection message assumes ledger-compactor is in the terminal Failed phase (terminal Pods are not counted) and that late-arrival and cargo-modeller have been deleted. If your figure differs, run `kubectl -n kcna-lab10 describe resourcequota mfs-lab-quota` and re-do the arithmetic - the *form* of the message is what matters: requested + used > limited.

* Node allocatable (CPU_ALLOC 8, MEM_ALLOC ~8Gi) reflects your Docker Desktop / Colima allocation and will differ. Nothing in this lab depends on it.

* kubectl top figures, if you installed metrics-server, are live measurements and differ every run. Idle Pods typically read 1m CPU and single-digit Mi of memory, so the HPA stays at minReplicas: 2.

* HPA TARGETS column width varies with terminal width; kubectl may wrap or truncate the two-metric string.

* If the HPA was created less than ~15 seconds before you read it, .status.conditions may be empty. The script accepts an empty ScalingActive reason for that reason; wait and re-run for the definitive line.